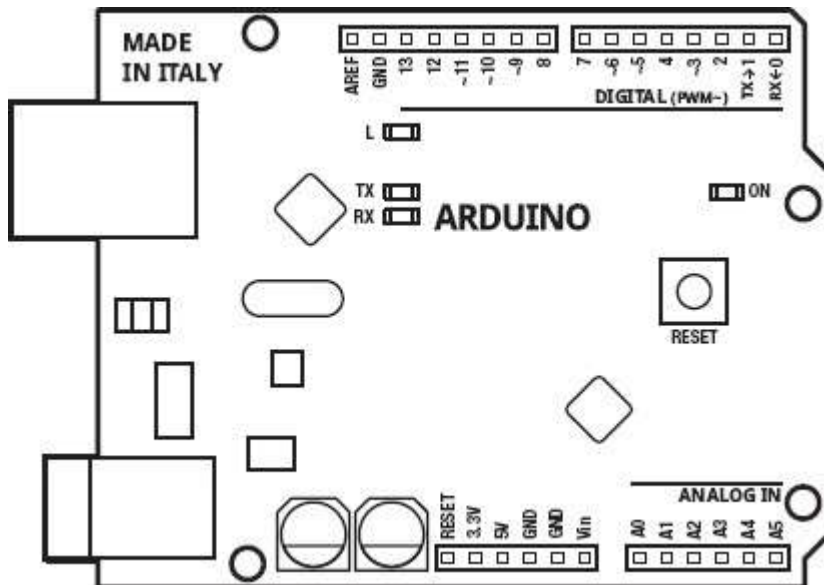


EXPERIMENT NO.1

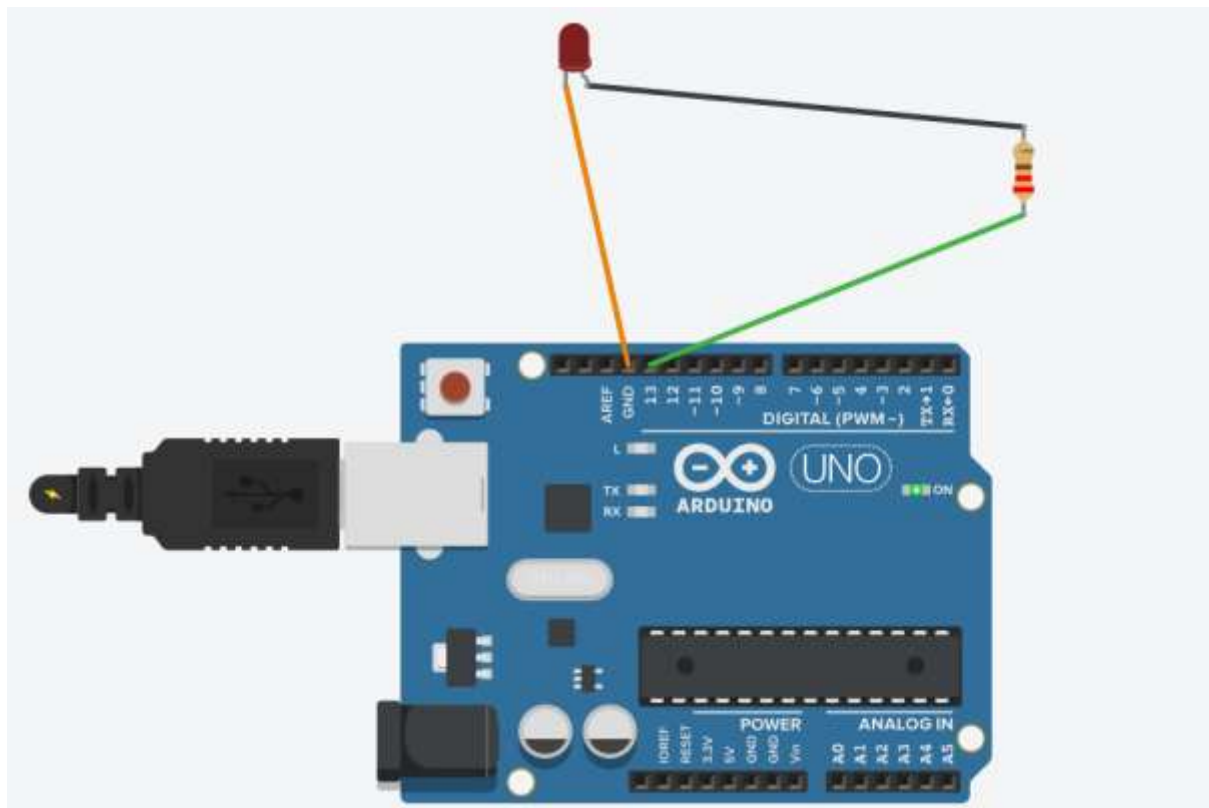
Interfacing of LEDs /switches with embedded core



1. Hardware Setup (1 LED, 1 Resistor)

Before opening the software, ensure your hardware is wired correctly. Using only one resistor means we are creating a single current loop.

- **Pin 13 (Signal):** Connect to one leg of your **220 Ω** resistor.
- **Resistor to LED:** Connect the other leg of the resistor to the **Anode** (the longer leg) of the LED.
- **GND (Ground):** Connect the **Cathode** (shorter leg) of the LED to any **GND** pin on the Arduino.



```
/*  
 * Experiment 1: LED Interfacing  
 * Setup: LED connected to Pin 13 with a series resistor.  
 */  
  
void setup() {  
  // Define Digital Pin 13 as an OUTPUT pin  
  pinMode(13, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(13, HIGH);    // Turn LED ON  
  delay(1000);               // Keep it ON for 1 second  
  digitalWrite(13, LOW);    // Turn LED OFF  
  delay(1000);               // Keep it OFF for 1 second  
}
```

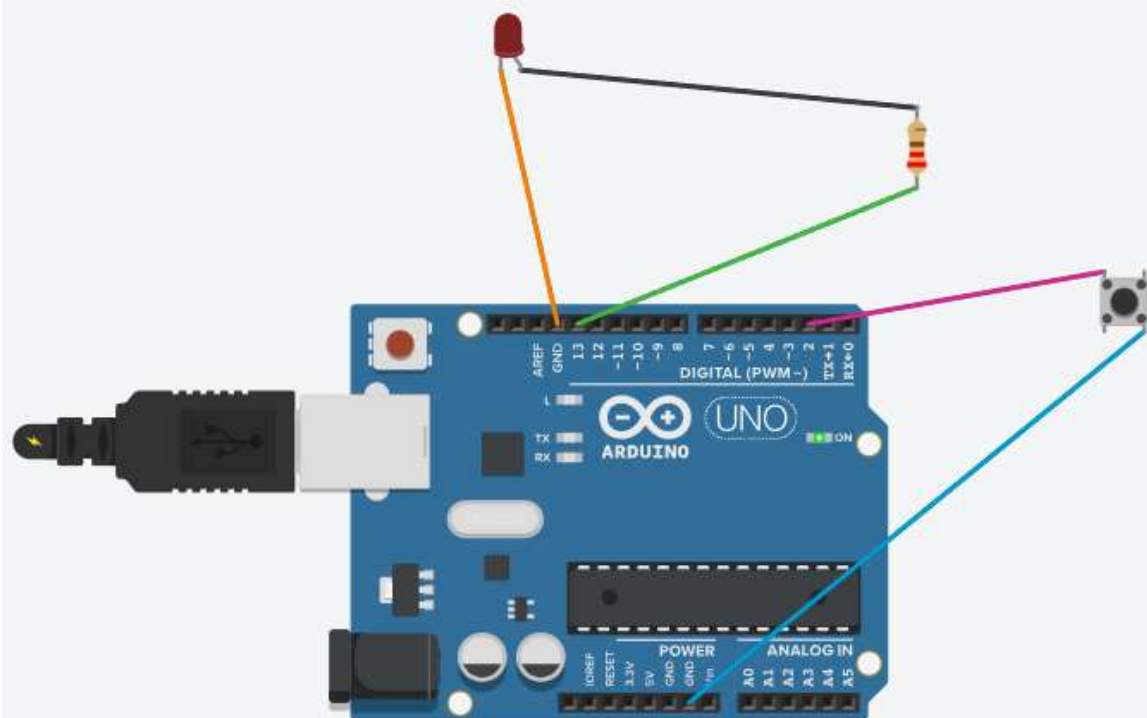
WITH SWITCH BUTTON:

1. The LED Path (Output)

- **Arduino Pin 13** → One end of the **Resistor**.
- **Other end of Resistor** → **Long leg (Anode)** of the LED.
- **Short leg (Cathode)** of the LED → **GND pin** (the one next to Pin 13).

2. The Switch Path (Input)

- **Arduino Pin 2** → One leg of the **Push button**.
- **Opposite leg of the Push button** → **GND pin** (the one on the bottom rail near the 5V pin).



```
const int ledPin = 13;    // Output pin for LED
const int buttonPin = 2;  // Input pin for Switch

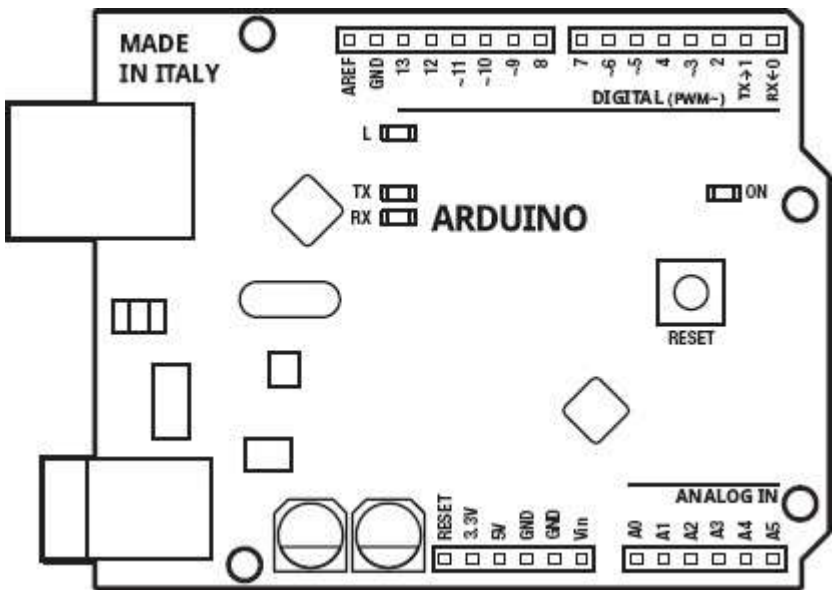
void setup() {
  // Initialize LED as Output
  pinMode(ledPin, OUTPUT);

  // Initialize Switch as Input with Internal Pull-up
  // This keeps the pin HIGH (5V) until the button is pressed
  pinMode(buttonPin, INPUT_PULLUP);
}

void loop() {
  // Read the digital state of the switch
  int buttonState = digitalRead(buttonPin);

  // Logic: Since we use PULLUP, LOW means the button is pressed
  if (buttonState == LOW) {
    digitalWrite(ledPin, HIGH); // LED ON
  } else {
    digitalWrite(ledPin, LOW);  // LED OFF
  }
}
```

Expt 2: Interfacing DHT 11 sensor with Arduino core





DHT sensor library by
Adafruit

1.4.7 installed

Arduino library for DHT11, DHT22,
etc Temp & Humidity Sensors
Arduino library for DHT11, DHT22,...
[More info](#)

1.4.7

REMOVE

2. Hardware Connection Guide	
Based on your documentation and the pin configuration table: 🔗	
DHT11 Pin	Arduino Connection
VCC	5V 🔗
DATA	Digital Pin 2 🔗
GND	GND 🔗
📄 Export to Sheets 📄	

```
#include <DHT.h>

#define DHTPIN 2          // Pin where DATA is connected [cite: 45]
#define DHTTYPE DHT11    // Defining the sensor type [cite: 46]

DHT dht(DHTPIN, DHTTYPE); // Initializing the sensor object [cite: 47]

void setup() {
  Serial.begin(9600); // Starting serial communication [cite: 49]
  dht.begin();        // Starting the sensor [cite: 50]
}

void loop() {
  // Reading humidity and temperature [cite: 54, 55]
  float h = dht.readHumidity();
  float t = dht.readTemperature();

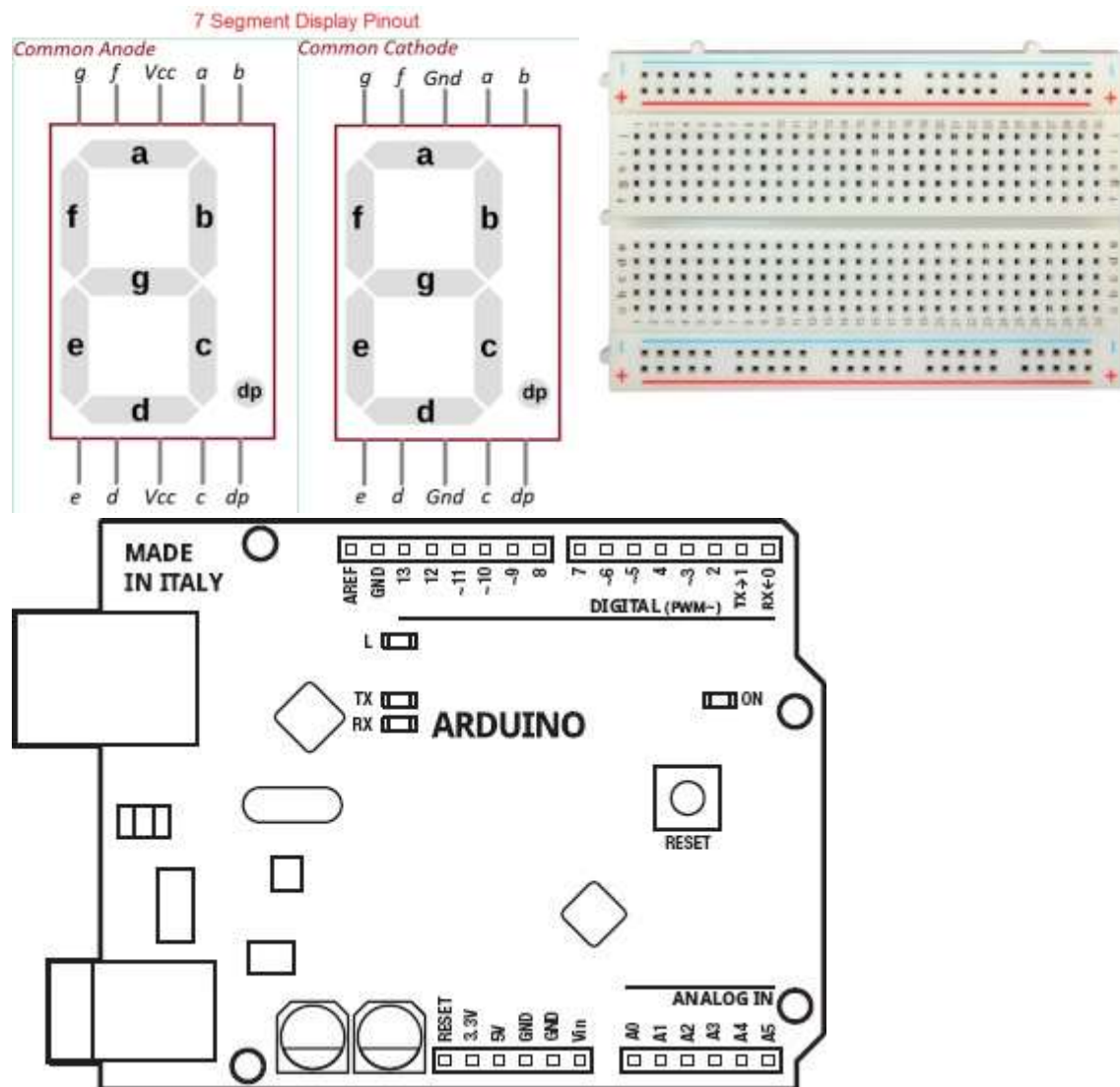
  // Basic check: if reading failed, skip this loop [cite: 56, 57]
  if (isnan(h) || isnan(t)) return;

  // Simple output format
  Serial.print("Temp: ");
  Serial.print(t);
  Serial.print(" C | Humidity: ");
  Serial.print(h);
  Serial.println(" %");

  delay(2000); // 2-second delay is required for DHT11 stability [cite: 67, 89]
}
```

EXPERIMENT NO.3

Interfacing Seven Segment Display with Arduino in Embedded core



1. Breadboard Connection Guide (Direct)

Segment	Display Pin	Arduino Pin
Common (Anode)	3 or 8	5V ↗ +1
a	7	Digital Pin 7 ↗ +1
b	6	Digital Pin 6 ↗ +1
c	4	Digital Pin 4 ↗ +1
d	2	Digital Pin 2 ↗ +1
e	1	Digital Pin 8 (Manual Pin 1 is TX, Pin 8 is used in code) ↗ +2
f	9	Digital Pin 9 ↗ +1
g	10	Digital Pin 10 ↗ +1
dp	5	Digital Pin 5 ↗ +1


```

// Pins based on your manual: a=7, b=6, c=4, d=2, e=8, f=9, g=10
const int segmentPins[] = {7, 6, 4, 2, 8, 9, 10};

// Patterns for 0-9 (Common Anode: 0 is ON, 1 is OFF) [cite: 134]
const byte digitPatterns[10] = {
    0b11000000, 0b11111001, 0b10100100, 0b10110000, 0b10011001,
    0b10010010, 0b10000010, 0b11111000, 0b10000000, 0b10010000
};

void setup() {
    for (int i = 0; i < 7; i++)
    {
        pinMode(segmentPins[i], OUTPUT); // [cite: 148, 149]
    }
}

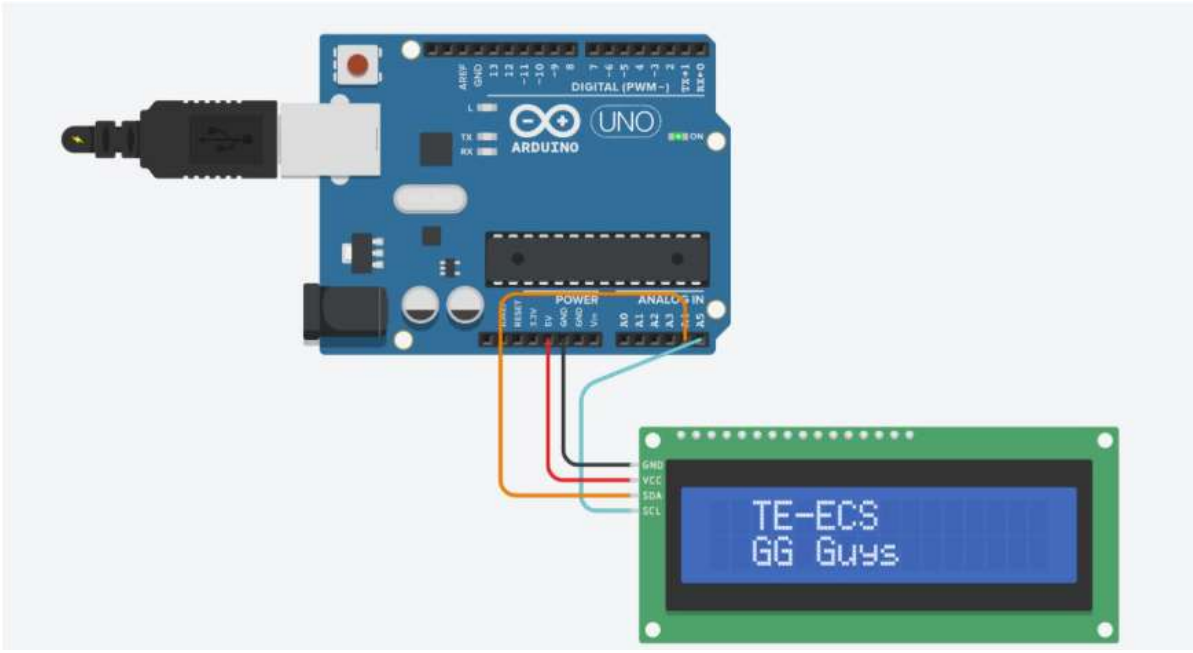
void loop() {
    // Countdown from 9 down to 1 [cite: 153]
    for (int digit = 9; digit >= 1; digit--) {
        displayDigit(digit);
        delay(1000); // 1 second interval [cite: 155]
    }
    // Loop restarts automatically; use the physical Reset button on Uno to restart manually
}

void displayDigit(int digit) {
    byte pattern = digitPatterns[digit]; // [cite: 159]
    for (int i = 0; i < 7; i++) {
        // Write LOW to turn segment ON (Common Anode logic) [cite: 161, 162]
        digitalWrite(segmentPins[i], (pattern & (1 << i)) ? HIGH : LOW);
    }
}

```

EXPERIMENT NO.4

Interfacing LCD 16 x 2 display using I2C module with arduino



2. Hardware Connection Guide		
Using the I2C module, follow these four connections to your Arduino Uno: 🔗		
LCD I2C Pin	Arduino Uno Pin	wire Color (Reference)
GND	GND	Black 🔗
VCC	5V	Red 🔗
SDA	A4	Orange 🔗
SCL	A5	Blue 🔗

```
#include <LiquidCrystal_I2C.h>

// Set the LCD address to 0x20 for a 16 chars and 2 line display
LiquidCrystal_I2C lcd(0x20, 16, 2);

void setup() {
  lcd.init();           // Initialize the LCD [cite: 211]
  lcd.clear();          // Clear any previous data [cite: 212]
  lcd.backlight();      // Turn on the backlight [cite: 213]

  lcd.setCursor(2, 0);  // Position: Column 2, Row 0 [cite: 214]
  lcd.print("TE-ECS"); // [cite: 215]

  lcd.setCursor(2, 1);  // Position: Column 2, Row 1 [cite: 216]
  lcd.print("GG Guys"); // [cite: 217]
}

void loop() {
  // Empty loop as we are displaying static text [cite: 219]
}
```

LiquidCrystal I2C by Frank de Brabander

1.1.2 installed

A library for I2C LCD displays. The library allows to control I2C displays with functions extremely similar to...

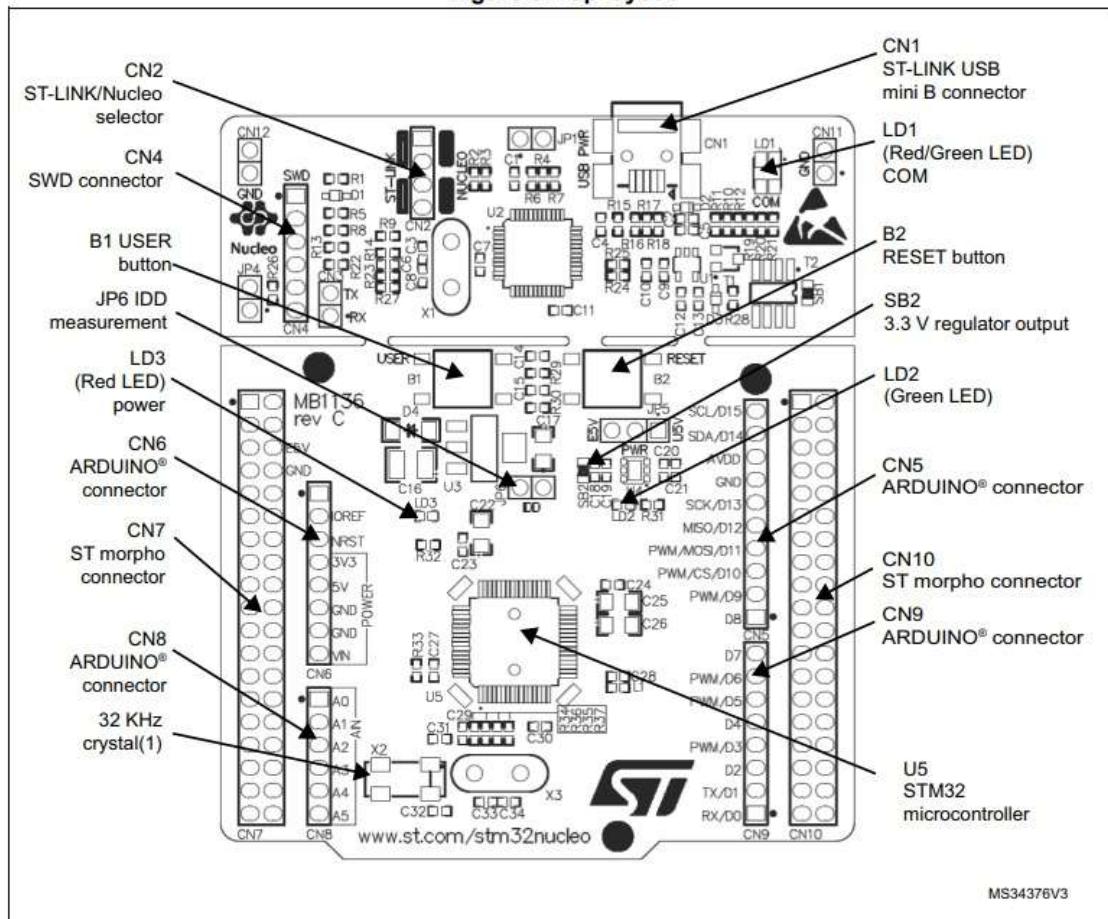
[More info](#)

1.1.2 ▼

REMOVE

EXPERIMENT NO.5

Interfacing and programming of Ultrasonic sensor with ARM based board STM 32



1. Proper Hardware Connections

Using your reference image of the Nucleo board, connect the HC-SR04 sensor to the following pins: [🔗 +2](#)

Ultrasonic Sensor Pin	Nucleo Board Physical Pin	Internal STM32 Name
VCC	5V (on CN6 Power Header)	5V Supply 🔗 +2
GND	GND (on CN6 Power Header)	Ground 🔗 +2
TRIG	A1 (on CN8 Analog Header)	PA1 (GPIO Output) 🔗 +2
ECHO	A2 (on CN8 Analog Header)	PA2 (GPIO Input) 🔗 +1

```
/* * ERTOS EXPERIMENT 5: Ultrasonic Sensor with STM32
 * Aim: Measure distance and display on Serial Monitor [cite: 231]
 */

// Define the pins based on your Nucleo reference [cite: 249]
const int trigPin = A1; // PA1
const int echoPin = A2; // PA2

void setup() {
    Serial.begin(9600);           // Initialize Serial Monitor
    pinMode(trigPin, OUTPUT);     // Set Trig as Output [cite: 262]
    pinMode(echoPin, INPUT);      // Set Echo as Input [cite: 263]
    Serial.println("Ultrasonic Sensor Initialized...");
}

void loop() {
    // 1. Clear the trigPin [cite: 294]
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2); [cite: 295]

    // 2. Send 10 microsecond pulse to trigger sensor [cite: 242, 266, 297]
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    // 3. Measure the duration of the Echo pulse [cite: 245, 246, 301]
    // pulseIn returns the time in microseconds
    long duration = pulseIn(echoPin, HIGH);
```

```
// 4. Calculate Distance in cm [cite: 247, 304]
// Formula: Distance = (Time * Speed of Sound) / 2
float distance = (duration * 0.0343) / 2;

// 5. Display the result [cite: 269, 309]
if (duration == 0) {
  Serial.println("Out of range!");
} else {
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
}

delay(500); // Wait half a second before next reading [cite: 292]
}
```

Experiment 7

Write a program to create multiple tasks and understand the task management feature of free RTOS.



2. Connection Table (Internal Logic)

Even though you aren't plugging in wires, the "connections" are happening internally as follows:

Component	Physical Location on Board	Internal Connection	Purpose
Green LED	Near the center (LD2)	PA5	Visual output for <code>BlinkTask</code> . 🔗
USB Port	Top center (CN1)	UART2	Data output for <code>SerialTask</code> . 🔗
Reset Button	Top right (B2)	NRST	Restarts the RTOS all tasks. 🔗

NO EXTERNAL CONNECTIONS REQUIRED

1) LED BLINK

2) SERIAL OUTPUT

STM32 FREERTOS

Type:

Topic:

STM32duino FreeRTOS by
Richard Barry...

10.3.3 installed

Real Time Operating System
implemented for STM32 FreeRTOS
provides methods for multiple...

[More info](#)

10.3.3

REMOVE

```
#include <Arduino.h>
#include <STM32FreeRTOS.h>

void BlinkTask(void *pvParameters);
void SerialTask(void *pvParameters);

void setup() {
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT); // Uses the onboard green LED

    // Creating tasks internally
    xTaskCreate(BlinkTask, "Blink", 128, NULL, 1, NULL);
    xTaskCreate(SerialTask, "Serial", 128, NULL, 1, NULL);

    vTaskStartScheduler(); // Start the real-time kernel [cite: 337]
}

void loop() {} // loop is empty; RTOS takes over [cite: 337]

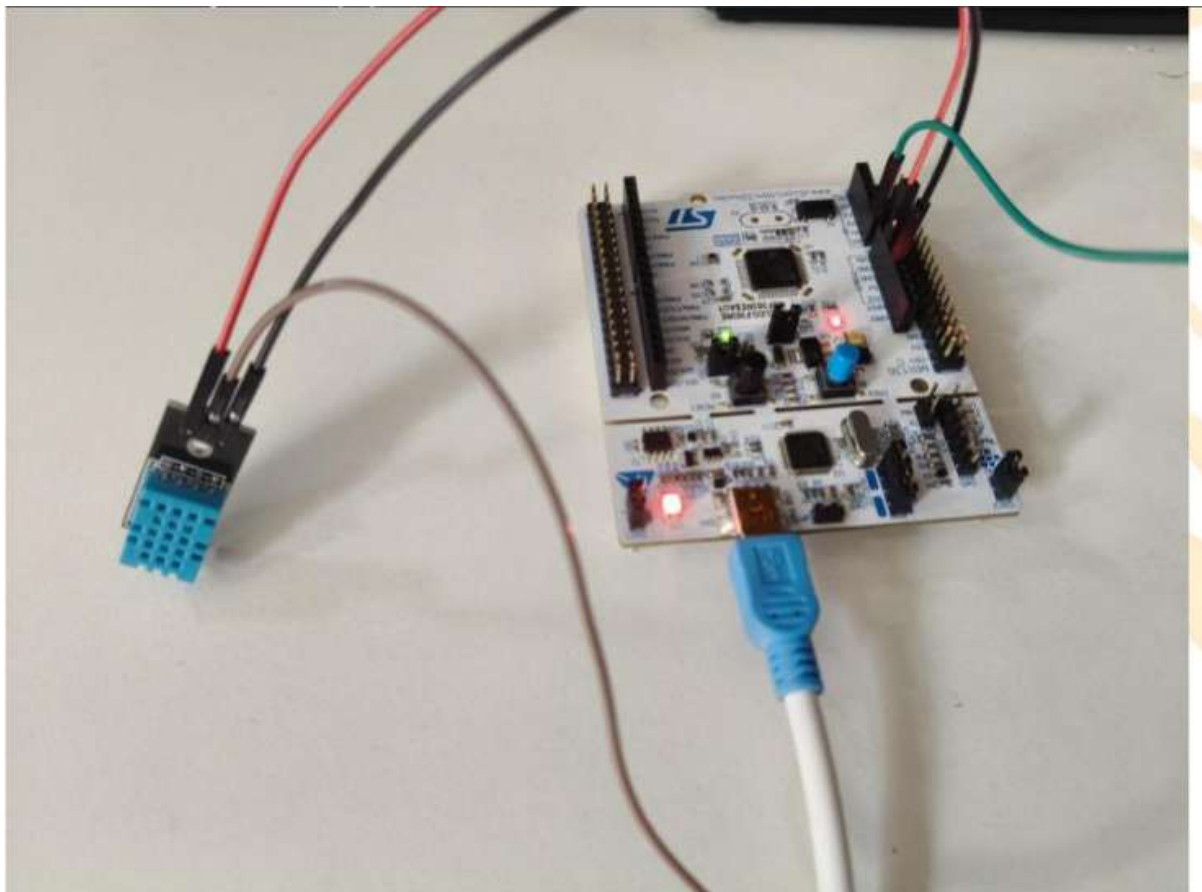
void BlinkTask(void *pvParameters) {
    for (;;) {
        digitalWrite(LED_BUILTIN, HIGH);
        vTaskDelay(500 / portTICK_PERIOD_MS); // Non-blocking delay [cite: 337]
        digitalWrite(LED_BUILTIN, LOW);
        vTaskDelay(500 / portTICK_PERIOD_MS);
    }
}
```



```
void SerialTask(void *pvParameters) {  
    for (;;) {  
        Serial.println("FreeRTOS Task Running"); // Output via USB  
        vTaskDelay(1000 / portTICK_PERIOD_MS);  
    }  
}
```

Experiment no - 8

AIM – Write a program to illustrate task scheduling feature of Free RTOS



Component	Physical Pin on Nucleo Header	internal STM32 Pin	Logic/Function
DHT11 VCC	5V (on CN6 Power rail)	5V Supply	Powers the sensor
DHT11 GND	GND (on CN6 or CN9)	Ground	Common Ground
DHT11 DATA	A0 (on CN8 Analog rail)	PA0	Digital Signal Input ↗ +1
Built-in LED	LD2 (on board)	PA5	Visual Status Output ↗
Serial Link	USB Mini-B Port	UART2	115200 Baud Data ↗

```
#include <STM32FreeRTOS.h>
#include "DHT.h"

// ----- CONFIG -----
#define LEDPIN PA5
#define DHTPIN PA0
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// Mutex for Serial to prevent data overlapping
SemaphoreHandle_t serialMutex;

// ----- TASKS -----

// Task 1: Toggles the built-in Green LED
void LEDTask(void *pvParameters) {
    pinMode(LEDPIN, OUTPUT);
    while (1) {
        digitalWrite(LEDPIN, !digitalRead(LEDPIN)); // Toggle
        vTaskDelay(pdMS_TO_TICKS(1000));           // 1 second delay
    }
}
```

```

// Task 2: Reads the DHT11 Sensor
void DHTTask(void *pvParameters) {
    while (1) {
        float temp = dht.readTemperature();
        float hum = dht.readHumidity();

        // Take Mutex to print safely
        if (xSemaphoreTake(serialMutex, portMAX_DELAY)) {
            if (isnan(temp) || isnan(hum)) {
                Serial.println("DHT Read Failed!");
            } else {
                Serial.print("Temp: ");
                Serial.print(temp);
                Serial.print(" C | Humidity: ");
                Serial.print(hum);
                Serial.println(" %");
            }
            xSemaphoreGive(serialMutex); // Release Mutex
        }
        vTaskDelay(pdMS_TO_TICKS(2000)); // Sensor needs 2s delay
    }
}

```

```

// Task 3: Simple system monitor print
void SerialTask(void *pvParameters) {
    while (1) {
        if (xSemaphoreTake(serialMutex, portMAX_DELAY)) {
            Serial.println("FreeRTOS Scheduler Running...");
            xSemaphoreGive(serialMutex);
        }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

// ----- SETUP -----
void setup() {
    Serial.begin(115200);
    dht.begin();

    // Create the Mutex
    serialMutex = xSemaphoreCreateMutex();

    // Create Tasks (Function, Name, Stack, Parameter, Priority, Handle)
    xTaskCreate(LEDTask, "LED", 128, NULL, 2, NULL);
    xTaskCreate(SerialTask, "Serial", 128, NULL, 1, NULL);
    xTaskCreate(DHTTask, "DHT", 256, NULL, 1, NULL);

    vTaskStartScheduler();
}

```

```

void loop() {
    // Empty - Scheduler takes over
}

```

Experiment no - 9

AIM – Write a program to illustrate Queue management feature of Free RTOS



Component	Physical Location	Purpose
STM32 MCU	Center of board (U5)	Runs the Sender and Receiver tasks and manages the Queue. 🔗 +1
USB Mini-B	Top of board (CN1)	Connects to your PC to show the "Sent" and "Received" messages. 🔗 +2
Onboard RAM	Inside the MCU	Acts as the physical storage for the Queue data. 🔗 +1

2. Key Queue Management Functions

Function	Purpose
<code>xQueueCreate()</code>	Initializes a new queue with a specific size and item length. ↗
<code>xQueueSend()</code>	Adds data to the back of the queue. ↗
<code>xQueueReceive()</code>	Reads and removes an item from the head of the queue. ↗
<code>xQueuePeek()</code>	Reads an item from the queue without removing it. ↗
<code>uxQueueMessagesWaiting()</code>	Checks how many items are currently stored in the queue. ↗

NO EXTERNAL CONNECTIONS

```
#include <Arduino.h>
#include <STM32FreeRTOS.h>

QueueHandle_t q; // Handle for the queue

// Sender Task: Puts data into the queue every 200ms
void sendTask(void *p) {
    int x = 0;
    while (1) {
        x++;
        xQueueSend(q, &x, 0); // Send the value of x to the queue
        Serial.print("Sent: ");
        Serial.println(x);
        vTaskDelay(200 / portTICK_PERIOD_MS);
    }
}

// Receiver Task: Reads data from the queue every 1000ms
void recvTask(void *p) {
    int y;
    while (1) {
        // Wait until data is available in the queue
        if (xQueueReceive(q, &y, portMAX_DELAY)) {
            Serial.print("Received: ");
            Serial.println(y);
            vTaskDelay(1000 / portTICK_PERIOD_MS);
        }
    }
}
```

```

void setup() {
    Serial.begin(115200);
    delay(2000); // Wait for Serial Monitor to open

    Serial.println("Queue FIFO Demo Initialized");

    // Create a queue that can hold 5 integers
    q = xQueueCreate(5, sizeof(int));

    // Create the two tasks
    xTaskCreate(sendTask, "Sender", 128, NULL, 1, NULL);
    xTaskCreate(recvTask, "Receiver", 128, NULL, 1, NULL);

    // Start the FreeRTOS Scheduler
    vTaskStartScheduler();
}

void loop() {
    // Empty: RTOS takes over
}

```

Output Serial Monitor X

Message (Enter to send message to 'Nucleo-64' on 'COM7')

```

Sent: 677
Sent: 678
Sent: 679
Sent: 680
Received: 656
Sent: 681
Sent: 682
Sent: 683
Sent: 684
Sent: 685
Received: 661
Sent: 686
Sent: 687
Sent: 688
Sent: 689
Sent: 690

```


Experiment no - 10

AIM – Write a program to illustrate the use of Binary and Counting Semaphore for Task Synchronization using Free RTOS.



NO EXTERNAL CONNECTIONS

1. Types of Semaphores

- **Binary Semaphores:**

- **Capacity:** Holds a single token (value 0 or 1). ⓘ +2
- **Purpose:** Primarily used for signaling between tasks or between an ISR and a task (e.g., waking a task when an interrupt occurs). ⓘ +1
- **Creation:** Initialized using `xSemaphoreCreateBinary()`. ⓘ

- **Counting Semaphores:**

- **Capacity:** Holds multiple tokens (value 0 to a specified Maximum). ⓘ +1
- **Purpose:** Used for managing resources with multiple instances (like a buffer pool) or tracking event occurrences. ⓘ +1
- **Creation:** Initialized using `xSemaphoreCreateCounting(max_count, initial_count)`. ⓘ

2. Core API Functions

- `xSemaphoreTake(xSemaphore, xBlockTime)` : Used to acquire a token. If none are available, the task can block for a specified time. [🔗 +2](#)
- `xSemaphoreGive(xSemaphore)` : Used to release or provide a token. [🔗 +1](#)
- `xSemaphoreGiveFromISR()` : A specialized version for releasing a semaphore safely from within an interrupt. [🔗 +1](#)

Part A: Binary Semaphore (Signaling)

In this setup, a **Sender** task "gives" a semaphore every second, signaling the **Receiver** task to blink the onboard LED. [🔗](#)

```

#include <STM32FreeRTOS.h>
#include <semphr.h>

SemaphoreHandle_t binSem;

void setup() {
    Serial.begin(115200);
    pinMode(LED_BUILTIN, OUTPUT);
    digitalWrite(LED_BUILTIN, HIGH); // LED OFF (Active Low)

    binSem = xSemaphoreCreateBinary(); // Create the semaphore

    xTaskCreate(TaskSender, "Sender", 128, NULL, 1, NULL);
    xTaskCreate(TaskReceiver, "Receiver", 128, NULL, 1, NULL);
    vTaskStartScheduler();
}

void TaskSender(void *pvParameters) {
    while (1) {
        Serial.println("Sender: Giving Semaphore");
        xSemaphoreGive(binSem); // Signal the receiver
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

```

```

void TaskReceiver(void *pvParameters) {
    while (1) {
        if (xSemaphoreTake(binSem, portMAX_DELAY)) { // Wait for signal
            Serial.println("Receiver: SUCCESS -> LED BLINK");
            digitalWrite(LED_BUILTIN, LOW); // LED ON
            vTaskDelay(pdMS_TO_TICKS(200));
            digitalWrite(LED_BUILTIN, HIGH); // LED OFF
        }
    }
}

void loop() {
    // This must remain empty as FreeRTOS handles the tasks
}

```

Part B: Counting Semaphore (Resource Management)

Here, a **Producer** adds resources (gives tokens) every 500ms, while a **Consumer** takes them to trigger an LED blink.  +1

```
#include <STM32FreeRTOS.h>
#include <semphr.h>

// Global Handles [cite: 471, 487]
SemaphoreHandle_t binSem;
SemaphoreHandle_t countSem;

void setup() {
    Serial.begin(115200);
    pinMode(LED_BUILTIN, OUTPUT);
    digitalWrite(LED_BUILTIN, HIGH); // LED OFF (Active LOW) [cite: 475]

    // Initialize Semaphores [cite: 476, 495]
    binSem = xSemaphoreCreateBinary();
    countSem = xSemaphoreCreateCounting(3, 0); // Max 3, Start at 0

    // Create Tasks [cite: 478, 502, 503]
    xTaskCreate(TaskProducer, "Prod", 128, NULL, 1, NULL);
    xTaskCreate(TaskConsumer, "Cons", 128, NULL, 1, NULL);

    vTaskStartScheduler(); // [cite: 479, 504]
}
```

```
// Producer Task: Gives both semaphores [cite: 481, 511]
void TaskProducer(void *pvParameters) {
    for (;;) {
        Serial.println("Producer: Adding Resource & Signaling");
        xSemaphoreGive(binSem);    // Signaling
        xSemaphoreGive(countSem);  // Resource management
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// Consumer Task: Takes semaphores and blinks LED [cite: 482, 519]
void TaskConsumer(void *pvParameters) {
    for (;;) {
        // Wait for either signaling or resource availability
        if (xSemaphoreTake(countSem, portMAX_DELAY)) {
            Serial.println("Consumer: SUCCESS -> LED BLINK");
            digitalWrite(LED_BUILTIN, LOW); // LED ON [cite: 522]
            vTaskDelay(pdMS_TO_TICKS(200)); // [cite: 523]
            digitalWrite(LED_BUILTIN, HIGH); // LED OFF [cite: 524]
        }
    }
}

void loop() {} // Required for Arduino compilation
```

BINARY

[illegible]

COUNTING

[illegible]